LECTURE 3

A) TRUSTWORTHINESS
B) COGNITIVE ARCHITECTURES FOR LANGUAGE AGENTS

JANUARY 12, 2026

DATASCI290: NEUROSYMBOLIC AI

TRUSTWORTHINESS

(IN AI SYSTEMS)

TRUSTWORTHY AI SYSTEMS

- What does it actually mean for an AI system to be trustworthy in high-consequence settings?
- Gaur, Manas, and Amit Sheth. *Building trustworthy NeuroSymbolic AI Systems: Consistency, reliability, explainability, and safety.* AI Magazine 45, no. 1 (2024): 139-155.

TRUST .. IS AN ENGINEERING PROBLEM

- In high-consequence domains: trust is not (primarily) about user satisfaction, UI polish, ….
- Trust means the system behaves *consistently*, *predictably*, and *in alignment* with domain obligations
- Errors are not just (acceptable) inaccuracies, they can translate directly into harm !
- Trust must be designed into the architecture
  - Not “patched on”

A MODEL OF TRUST, OR RATHER **THE** MODEL

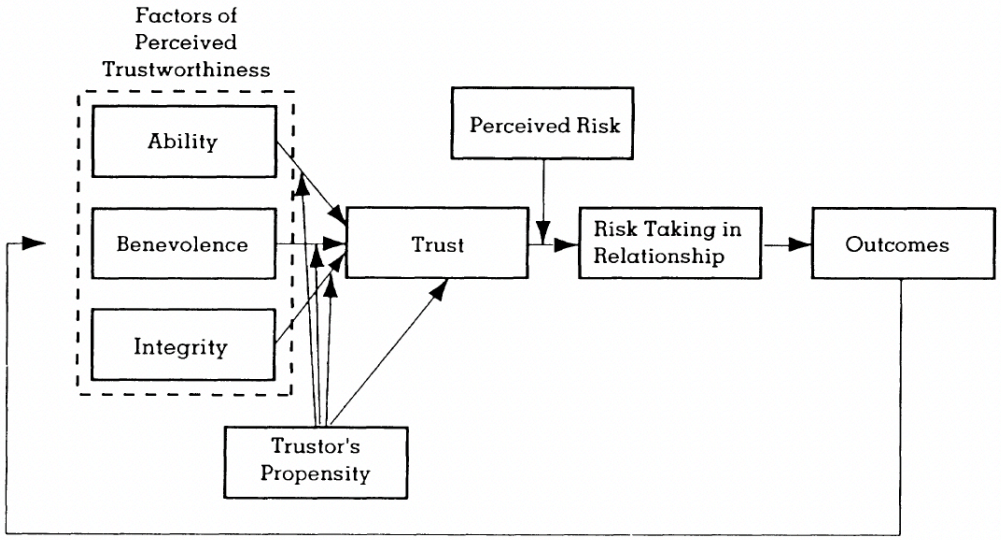
Academy of Management Review
1995, Vol. 20 No. 3, 709–734.

AN INTEGRATIVE MODEL OF ORGANIZATIONAL TRUST

ROGER C. MAYER
JAMES H. DAVIS
University of Notre Dame
F. DAVID SCHOORMAN
Purdue University

Scholars in various disciplines have considered the causes, nature, and effects of trust. Prior approaches to studying trust are considered, including characteristics of the trustor, the trustee, and the role of risk. A definition of trust and a model of its antecedents and outcomes are presented, which integrate research from multiple disciplines and differentiate trust from similar constructs. Several research propositions based on the model are presented.

FIGURE 1
Proposed Model of Trust



TRUST **ANTECEDENTS**

TABLE 1
Trust Antecedents

Authors	Antecedent Factors
Boyle & Bonacich (1970)	Past interactions, index of caution based on prisoners' dilemma outcomes
Butler (1991)	Availability, competence, consistency, discreetness, fairness, integrity, loyalty, openness, promise fulfillment, receptivity
Cook & Wall (1980)	Trustworthy intentions, ability
Dasgupta (1988)	Credible threat of punishment, credibility of promises
Deutsch (1960)	Ability, intention to produce
Farris, Senner, & Butterfield (1973)	Openness, ownership of feelings, experimentation with new behavior, group norms
Frost, Stimpson, & Maughan (1978)	Dependence on trustee, altruism
Gabarro (1978)	Openness, previous outcomes
Giffin (1967)	Expertness, reliability as information source, intentions, dynamism, personal attraction, reputation
Good (1988)	Ability, intention, trustees' claims about how (they) will behave
Hart, Capps, Cangemi, & Caillouet (1986)	Openness/congruity, shared values, autonomy/feedback
Hovland, Janis, & Kelley (1953)	Expertise, motivation to lie
Johnson-George & Swap (1982)	Reliability
Jones, James, & Bruni (1975)	Ability, behavior is relevant to the individual's needs and desires
Kee & Knox (1970)	Competence, motives
Larzelere & Huston (1980)	Benevolence, honesty
Lieberman (1981)	Competence, integrity
Mishra (In press)	Competence, openness, caring, reliability
Ring & Van de Ven (1992)	Moral integrity, goodwill
Rosen & Jerdee (1977)	Judgment or competence, group goals
Sitkin & Roth (1993)	Ability, value congruence
Solomon (1960)	Benevolence
Strickland (1958)	Benevolence

BACKGROUND **KNOWLEDGE,** AS A FIRST-CLASS CITIZEN

- Symbolic structures encode domain concepts, relationships, and processes
- This includes ontologies, knowledge graphs, and procedural knowledge
- Such structures make reasoning explicit and inspectable
- They are the substrate on which explainability and verification are built

THE **CREST** FRAMEWORK

- Trustworthiness: **four fundamental properties**
  - **Consistency**
  - **Reliability**
  - **Explainability**
  - **Safety**
- Each property is a concrete system behavior
- Each property corresponds to specific architectural requirements

CONSISTENCY

- A system is consistent if it produces the same decision for semantically equivalent inputs
- This includes paraphrases, rephrasings, and minor variations in wording
- Inconsistent behavior undermines user trust and can lead to contradictory actions
- In healthcare or safety contexts, inconsistency is itself a form of error
- LLMs are *inherently* inconsistent
  - Sensitive to surface form and token-level variation
  - Rely on statistical patterns rather than concept grounding
  - May make different assumptions depending on phrasing
  - Without explicit concepts, semantic equivalence is not enforced

HOW NEUROSYMBOLIC SYSTEMS ADDRESS CONSISTENCY

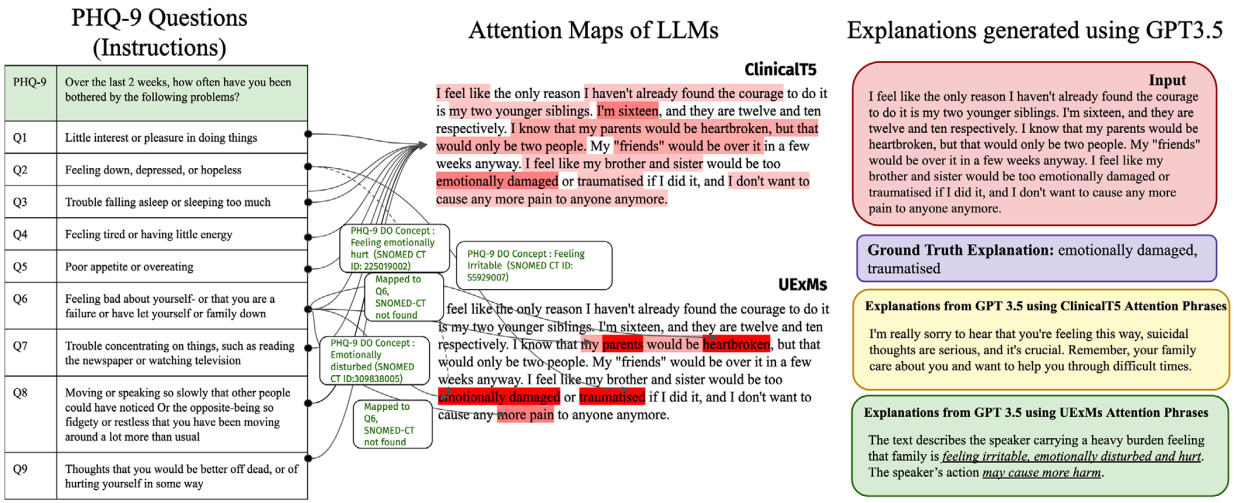
- Mapping inputs to canonical concepts using ontologies
- Reasoning over concept identifiers rather than raw text
- Applying the same rules and constraints to equivalent situations
- This enforces invariance at the knowledge level

RELIABILITY

- Reliability is about *predictable* correctness, under *varying* conditions
- It includes robustness to noise, ambiguity, and adversarial inputs
- High accuracy on benchmarks does **not** imply reliability in deployment !
- Reliability must be engineered, through
  - Redundancy
  - Verification

KNOWLEDGE-INFUSED ENSEMBLING, FOR RELIABILITY

- Using *ensembles of models* guided by external knowledge
- Instead of majority voting, knowledge coherence is used as a signal
- Outputs are evaluated against domain constraints and semantic relatedness
- This **turns knowledge into an active participant** in decision making



EXPLAINABILITY

- **System-level** explainability focuses on internal model mechanics
- **User-level** explainability focuses on domain-relevant reasons
- Users need explanations in their own conceptual language
- In medicine, this means clinical concepts, not attention maps

USER-LEVEL EXPLAINABLE MODELS (UEXMS)

- Generator–Evaluator architectures
- The generator produces candidate responses
- The evaluator checks them against domain knowledge and process rules
- Only responses that satisfy constraints are presented to the user

WHY POST-HOC EXPLANATIONS ARE NOT ENOUGH

- Post-hoc explanations rationalize after the fact
- They do not guarantee the decision was made correctly
- User-level explainability must be built into the reasoning process
- This requires explicit knowledge and rules during generation

SAFETY
(MORE THAN CONTENT FILTERING)

- Safety is often treated as a moderation problem
- In consequence-aware systems, safety is a reasoning problem
- The system must recognize risky states and respond appropriately
- This requires domain models of harm, escalation, and obligation

PROCESS KNOWLEDGE AND SAFETY

- Knowledge: *procedural* knowledge, and *process* knowledge
  - Procedural knowledge: knowing how to do something
  - Process knowledge: knowing how a process unfolds over time
- This includes guidelines, workflows, and decision trees
- For example, clinical escalation protocols in mental health
- These define what must happen, not just what may happen

PKIL: PROCESS KNOWLEDGE INFUSED LEARNING

- PKIL integrates process knowledge into the learning loop
- The model is guided not just by data but by procedural constraints
- This shapes the space of allowable behaviors
- It is a way of embedding obligations into learning

ALIGNMENT REVISITED

- Alignment is not just value matching
- It includes respecting domain constraints and obligations
- Misalignment can arise from missing context, not malicious intent
- Symbolic knowledge provides the missing structure
- Essentially, LLMs fail not because they are (inherently) unethical, but because they are **unstructured**

WHY THIS MATTERS FOR CONSEQUENCE-AWARE AGENTS

- Agents act, not just answer
- They trigger workflows, recommendations, and interventions
- In such systems, reasoning errors propagate into the world
- This is why trustworthiness is non-negotiable

FROM ARCHITECTURE TO PRACTICE

- Neurosymbolic design choices determine system behavior
- They enable auditing, verification, and accountability
- They also make regulatory compliance possible
- Without structure, there is nothing to certify

CREST WORK: SUMMARY

- Trust is an architectural property, not a cosmetic feature
- Neurosymbolic AI provides the structures required for accountability
- The CREST framework articulates concrete engineering goals
- This is the foundation for building consequence-aware cognitive agents

LANGUAGE AGENTS

COGNITIVE ARCHITECTURES
(FOR LANGUAGE AGENTS)

COGNITIVE AGENT ARCHITECTURES

- Our interest is in **building cognitive agents**
- Let us step through the ideas propounded on architectures for cognitive agents in this (rather thoughtfully put together !) paper
  - Sumers, T., Yao, S., Narasimhan, K.R. and Griffiths, T.L., 2023. Cognitive architectures for language agents. *Transactions on Machine Learning Research*. arXiv: https://arxiv.org/abs/2309.02427

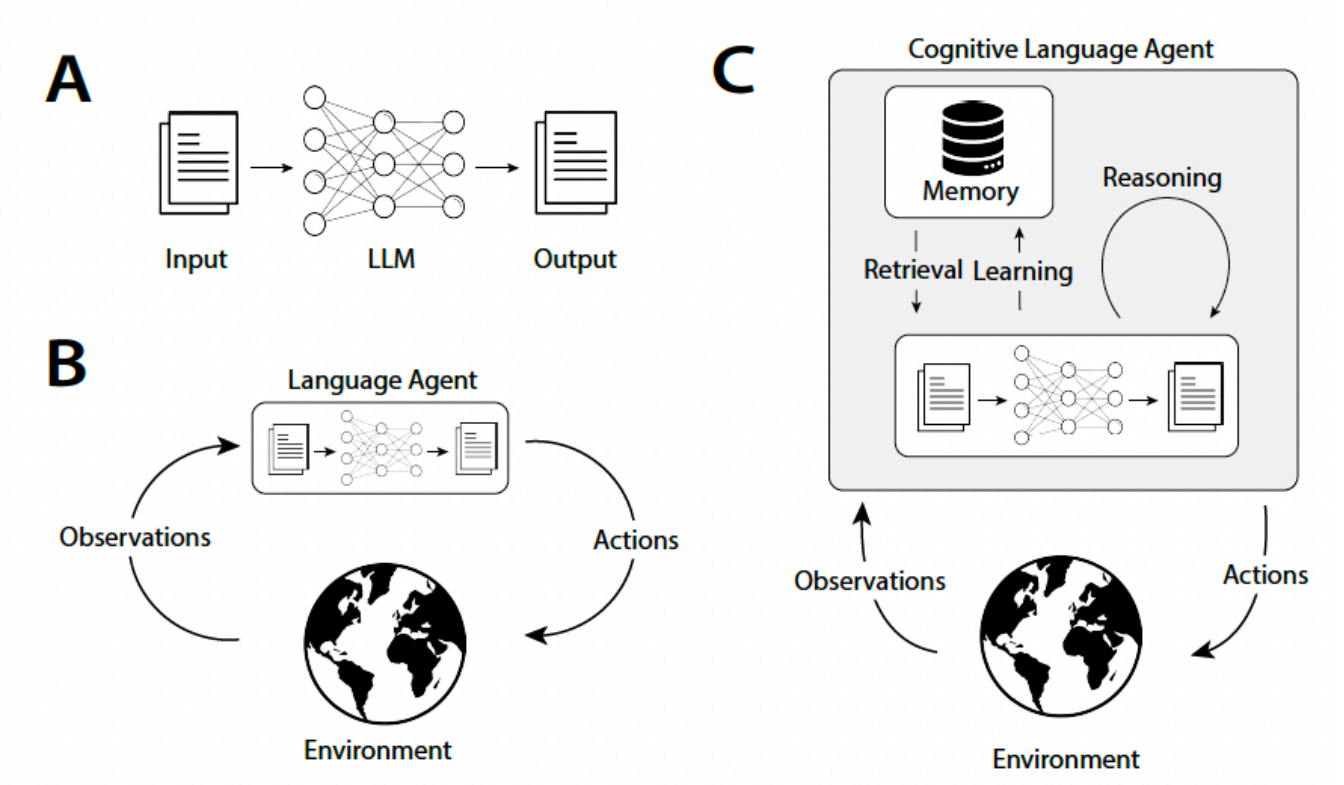
FROM SYMBOLS TO ALGORITHMS TO AGENTS

- Early view: cognition as **symbol manipulation**
  - Newell & Simon: human problem solving as manipulation of symbolic structures
  - Knowledge represented explicitly; reasoning as rule application over symbols
- **Algorithms** add control flow
  - Conditionals, loops, subroutines give structure to symbolic processing
  - Control determines which rules apply and in what order
- **Agents** add embodiment and environment
  - Perception-action loop closes the system
  - Cognition becomes continuous interaction, not one-shot computation

LANGUAGE AGENTS

- **Language Agents**: an emerging class of AI agents
  - Applying to the advances and power of language models (GenAI/LLMs) to building agents
- LLMs, powerful as they are, have limted
  - Knowledge
  - Reasoning

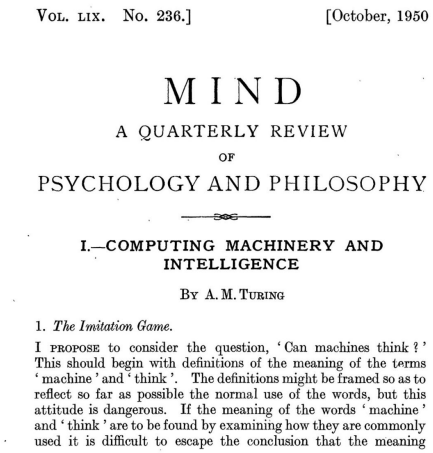
LLMS VS LANGUAGE AGENTS VS COGNITIVE LANGUAGE AGENTS



COALA

- CoALA: Cognitive Architectures for Language Agents
  - *“a conceptual framework to characterize and design general purpose language agents”*
- Along **three** dimensions
  - Information storage
    - working, long-term
  - Action space
    - internal, external
  - Decision making procedure

1932-36: COMPUTATION - THE PHILOSOPHICAL AND MATHEMATICAL EARTHQUAKE



Vol. lix. No. 236.] [October, 1950

MIND

A QUARTERLY REVIEW

OF

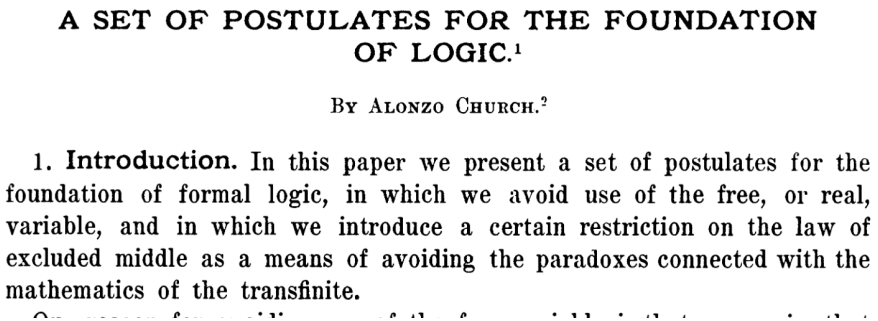
PSYCHOLOGY AND PHILOSOPHY

I.—COMPUTING MACHINERY AND INTELLIGENCE

By A. M. Turing

1. *The Imitation Game.*

I propose to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words 'machine' and 'think' are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning



A SET OF POSTULATES FOR THE FOUNDATION OF LOGIC.[1]

By Alonzo Church.[2]

1. **Introduction.** In this paper we present a set of postulates for the foundation of formal logic, in which we avoid use of the free, or real, variable, and in which we introduce a certain restriction on the law of excluded middle as a means of avoiding the paradoxes connected with the mathematics of the transfinite.

- Essentially: Anything that can be calculated by a systematic, mechanical procedure can be calculated by a formal mathematical system
- Turing Machine introduced shortly after Church's thesis
- Independently arrived at by
  - Church (lambda calculus)
  - Turing (Turing machines)
  - Gödel (recursive functions)
- In simple terms
  - If (some) reasoning is rule-based, it is mechanizable
  - Human reasoning can, in principle, be formalized

PRODUCTION SYSTEMS: THE CORE ENGINE

- A **production system** is
  - A set of IF–THEN rules operating over a working memory
  - IF matches current state; THEN proposes or executes an action
  - For instance
    - XYZ —> XWZ
    - W —> AB
    - ….
- Why production systems were revolutionary
  - Unified representation of knowledge and control
  - Behavior emerges from rule interaction, not fixed programs
- Key mechanisms
  - Pattern matching: which rules apply now?
  - Conflict resolution: which rule fires?
  - Execution: modify memory or act in the world

THERMOSTAT EXAMPLE

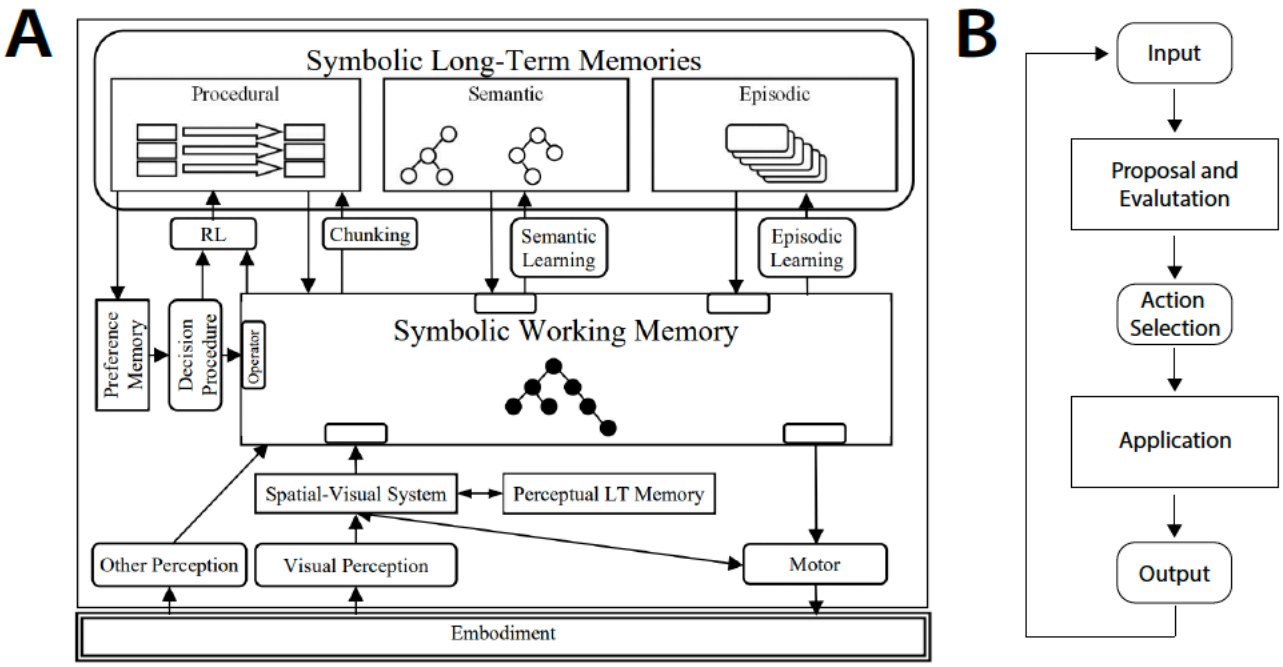
$$
\begin{aligned}
(\text{temperature} > 70^\circ) \wedge (\text{temperature} < 72^\circ) &\rightarrow \text{stop} \\
\text{temperature} < 32^\circ &\rightarrow \text{call for repairs; turn on electric heater} \\
(\text{temperature} < 70^\circ) \wedge (\text{furnace off}) &\rightarrow \text{turn on furnace} \\
(\text{temperature} > 72^\circ) \wedge (\text{furnace on}) &\rightarrow \text{turn off furnace}
\end{aligned}
$$

- Implications
  - Observe temperature → apply rule → turn heater on/off
  - Demonstrates perception, rule application, and action in a loop
- Essentially
  - Even trivial agents require state, rules, and control
  - Scaling up adds complexity, not new kinds of logic

FROM PRODUCTION SYSTEMS TO COGNITIVE ARCHITECTURES

- Why production systems were not enough
  - No built-in perception, learning, or long-term memory
  - Difficult to scale to complex, real-world tasks
- Cognitive architectures added structure
  - Working memory + long-term memory
  - Explicit decision procedures
  - Learning mechanisms
- Examples
  - Soar, ACT-R, and others
  - Designed to model human cognition and build complete agents

SOAR



- Laird, John E., and Paul S. Rosenbloom. "The evolution of the Soar cognitive architecture." In *Mind matters*, pp. 1-50. Psychology Press, 2014.

CONTROL FLOW AS THE HEART OF COGNITION

- Why control flow matters
  - Determines when to reason, when to act, when to learn
  - Separates random behavior from purposeful behavior
- Control in production systems
  - Rule matching and conflict resolution implement control
  - State of working memory guides execution path
- Control in agents
  - Task decomposition
  - Goal management
  - Subgoal creation and resolution

SOAR

- Memory
- Grounding
- Decision making
- Learning

ENTER LANGUAGE MODELS – WHAT CHANGED

- What LLMs provide
  - Powerful pattern recognition over language
  - Implicit knowledge learned from massive data
  - Ability to generate plans, explanations, and code
- What LLMs do not provide
  - Explicit control flow
  - Guaranteed correctness
  - Stable long-term memory
  - Hard constraints

LANGUAGE MODELS AS PROBABILISTIC PRODUCTION SYSTEMS

- The core analogy
  - Production rule: IF state THEN action
  - LLM: given prompt (state), sample next token/action
  - Both map state to action
- Why 'probabilistic' matters
  - Multiple possible actions with different probabilities
  - No guarantee the 'right' rule fires
  - Repeatability is not assured
- Implication
  - Great flexibility
  - Poor reliability for high-consequence decisions

PROMPT ENGINEERING AS CONTROL FLOW

- Prompts shape behavior
  - What you include determines what the model attends to
  - Few-shot examples bias rule selection
  - Instructions act as soft constraints
- Chaining prompts
  - Implements multi-step reasoning
  - Each prompt-response pair is a production step
- Why this is fragile
  - No hard guarantees
  - Small wording changes can change behavior

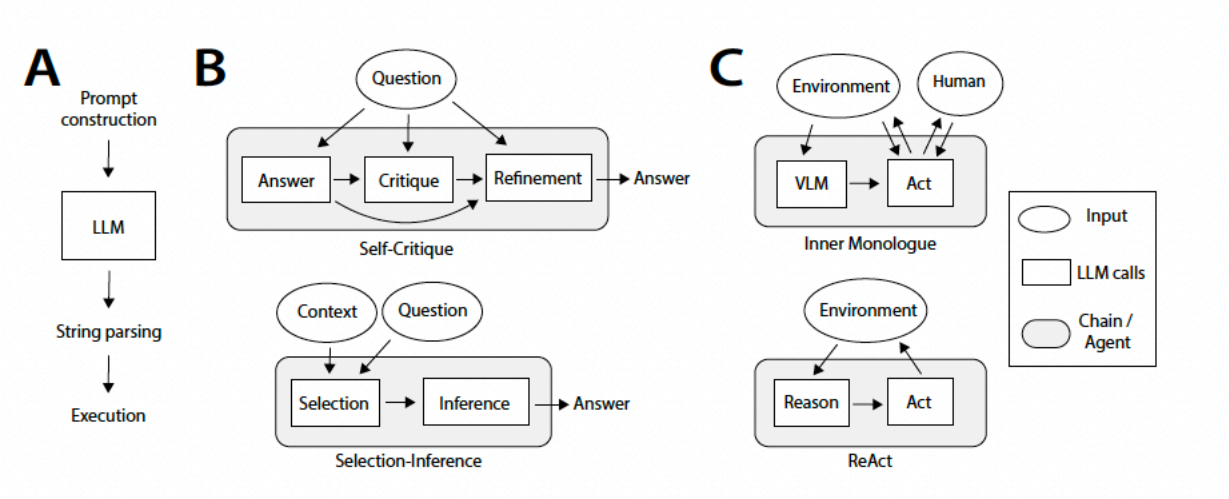
FROM SINGLE CALLS TO AGENT LOOPS

- Single LLM call
  - Input → output
  - No memory, no environment interaction
- Agent loop
  - Observe → think → act → observe → repeat
  - State persists across steps
- Why loops matter
  - Enables planning, correction, and adaptation
  - Moves from answering to acting

PROMPTING METHODS AND PRODUCTION SEQUENCES

Prompting Method	Production Sequence
Zero-shot	$Q \overset{\text{LLM}}{\rightsquigarrow} Q\ A$
Few-shot	$Q \longrightarrow Q_1\ A_1\ Q_2\ A_2\ Q \overset{\text{LLM}}{\rightsquigarrow} Q_1\ A_1\ Q_2\ A_2\ Q\ A$
Retrieval Augmented Generation	$Q \overset{\text{Wiki}}{\longrightarrow} Q\,O \overset{\text{LLM}}{\rightsquigarrow} Q\,O\,A$
Socratic Models	$Q \overset{\text{VLM}}{\rightsquigarrow} Q\,O \overset{\text{LLM}}{\rightsquigarrow} Q\,O\,A$
Self-Critique	$Q \overset{\text{LLM}}{\rightsquigarrow} Q\,A \overset{\text{LLM}}{\rightsquigarrow} Q\,A\,C \overset{\text{LLM}}{\rightsquigarrow} Q\,A\,C\,A$

PROMPTING METHOD, AND PRODUCTION SEQUENCES



- Essentially, modern LLM-based systems are gradually reconstructing classical cognitive control structures
  - Using prompts, chains, and agent loops
- Types
  - A = primitive, linear execution
  - B = internal reasoning loops
  - C = full agent interaction with environment

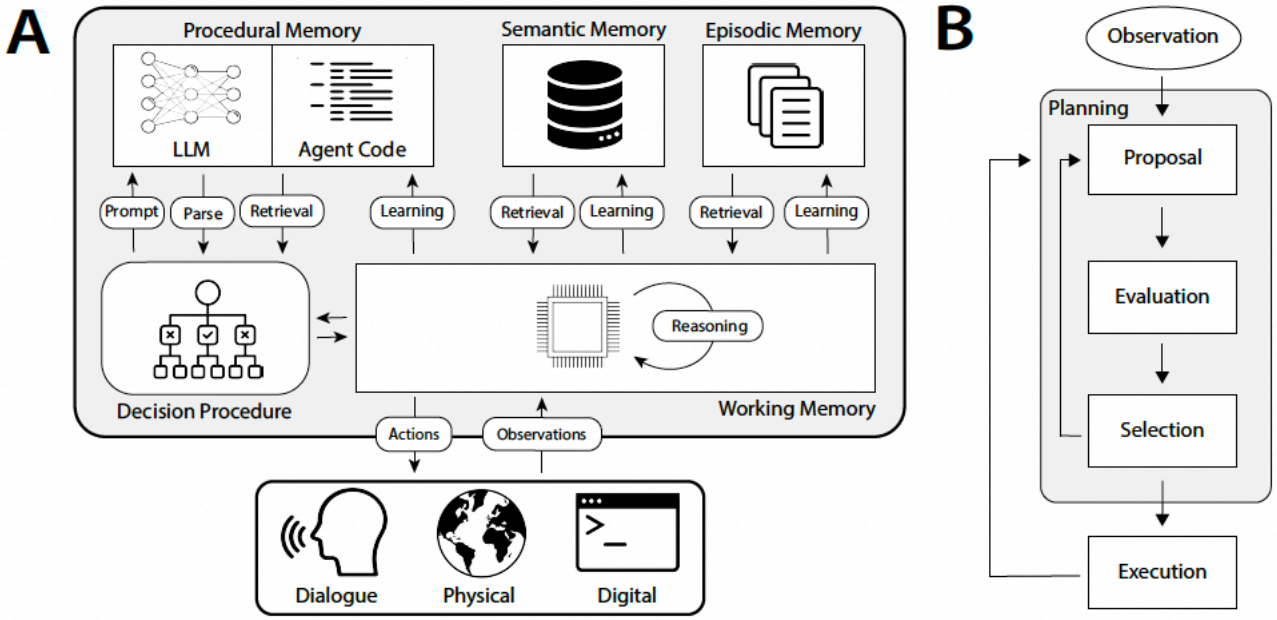
WHY 'COGNITIVE' AGENTS ARE DIFFERENT

- Reactive agents
  - Respond to observations
  - Little internal structure
- Cognitive agents
  - Have internal goals, plans, and memories
  - Reason about their own state
  - Learn from experience
- Key shift
  - From stimulus-response to deliberation

COALA – THE ARCHITECTURAL BLUEPRINT

- What CoALA proposes
  - A modular architecture for cognitive language agents
  - Explicit memory systems
  - Explicit action types
  - Explicit decision procedure
- Why an architecture is needed
  - To make behavior predictable
  - To enable inspection and control
  - To support learning over time

COALA: AGENT ARCHITECTURE



MEMORY IN COALA

- Working memory
  - Current task state
  - Goals, constraints, intermediate results
- Semantic memory
  - Facts, concepts, domain knowledge
- Episodic memory
  - Specific past experiences
- Procedural memory
  - Skills, routines, code, methods

WHY MEMORY IS CENTRAL

- Without memory
  - Agents forget past actions
  - Cannot build experience
  - Cannot improve over time
- With memory
  - Agents accumulate knowledge
  - Agents develop skills
  - Agents become more reliable

ACTION TYPES IN COALA

- Internal actions
  - Reasoning
  - Retrieval
  - Learning
- External actions
  - Tool use
  - API calls
  - Physical actions (real or simulated)
- Why the distinction matters
  - Internal actions change the agent
  - External actions change the world

GROUNDING – CONNECTING TO REALITY

- What grounding is
  - Linking symbols to real-world data or actions
  - APIs, sensors, databases, humans
- Why grounding is critical
  - Prevents hallucination
  - Enables verification
  - Provides feedback

RETRIEVAL – ACCESSING KNOWLEDGE

- Retrieval as an action
  - Agent decides when and what to retrieve
  - Not automatic; part of control flow
- Sources
  - Semantic memory
  - Episodic memory
  - External stores (databases, KGs, documents)

REASONING – BUILDING INTERNAL STRUCTURE

- What reasoning actions do
  - Decompose tasks
  - Generate candidate plans
  - Critique options
- Why reasoning alone is insufficient
  - Still probabilistic
  - Can be wrong with confidence

LEARNING – CHANGING THE AGENT

- Semantic learning
  - Adding new facts or correcting beliefs
- Episodic learning
  - Storing experiences
- Procedural learning
  - Acquiring new skills or routines
- Risk
  - Bad learning hardens errors

DECISION CYCLE: PROPOSE, EVALUATE, SELECT, EXECUTE

- Propose
  - Generate possible actions
- Evaluate
  - Estimate value, risk, or utility
- Select
  - Choose one action
- Execute
  - Perform action and update state

EXPLICIT DECISION STRUCTURE MATTERS

- Debuggability
  - We know where errors enter
- Safety
  - We can insert checks before execution
- Governance
  - We can audit decisions

COALA: AGENT EXAMPLES

- ReAct
  - Reasoning + acting loop
  - Minimal memory
- Voyager
  - Procedural memory growth (code)
- Generative Agents
  - Episodic + semantic memory

WHAT COALA SOLVES

- Brings structure back
  - Explicit memory
  - Explicit control
  - Explicit decision cycle
- Unifies classical AI and modern LLMs
  - Rules + learning
  - Symbols + neural models

WHAT COALA DOES NOT SOLVE

- Hard guarantees
  - Still probabilistic
- Formal verification
  - Needs symbolic layers
- Ethics and policy
  - Must be added explicitly

TODAY

- From models that generate plausible text to systems that can be trusted when consequences matter
  - The technical bar changes from “high accuracy” to “reliable behavior under constraints”
- Two complementary ideas emerged
  - Trust is an engineering property: consistency, traceability, and accountability have to be designed in
  - Agency is an architectural property: capable behavior comes from explicit memory, control, and feedback loops, not a single model call
- A unifying scientific takeaway
  - Many modern “prompting methods” are rediscovering classical control patterns: retrieve, propose, evaluate, revise, act, learn
  - The difference between a demo and a dependable system is whether those patterns are explicit, testable, and constrained
- We will treat knowledge, rules/obligations, and verification as first-class components of an agent architecture